



The Design and Implementation of a VLSI Chess Move Generator

Carl Ebeling
Andrew Palay

Department of Computer Science
Carnegie-Mellon University
Pittsburgh, Pennsylvania 15213

Abstract

Communication is a basic problem when using VLSI technology to implement large parallel circuits. Valuable chip area must be used to run wires connecting components on a chip and current packaging technology restricts the amount of communication that can cross chip boundaries. This paper presents a large parallel architecture for generating moves in chess and shows how it can be restructured to reduce communication and permit a straightforward VLSI implementation without any performance loss. The result is a move generator comprising 64 identical custom chips performing at a rate of 500,000 moves per second, performance that is comparable to the best existing move generator.

The success of the architecture of a component module like a chess move generator depends not only on its performance but on how well it meshes with the rest of the system. We discuss the requirements of a chess move generator in the context of a chess-playing system and describe how each of these are met by our design. Details of the chip design are presented along with a description of how the move generator is built using identical chips.

1. Introduction

The development of VLSI technology offers tremendous possibilities for constructing large parallel circuits. Unfortunately, the problem of communication can place strong restrictions on the architectures that can actually be built. Large scale parallel processing usually implies large communication bandwidth and communication is generally very expensive in VLSI implementations whereas circuit devices are cheap.

This paper describes a novel parallel architecture for a chess move generator. In this architecture, a relatively small number of extra devices are added in order to greatly reduce the communication costs. This architecture is interesting for several reasons. It is effective: in terms of absolute performance, it matches or exceeds all other current designs. It includes features that make it an integral part of a chess-playing system so that it is possible to actually use the performance in a real system. Finally, it is an architecture that lends itself to VLSI implementation. By reducing the communication costs, it is possible to use VLSI technology to build the major components of the move generator.

This paper starts with a general discussion of the problem of chess move generation. A high performance move generator must interface well with the chess-playing algorithm; otherwise, much of the performance gain may be lost. The α - β search algorithm² makes very specific demands on the way moves are to be generated and we examine how these demands are satisfied. We next examine two previous hardware designs for chess move generators, indicating the problems associated with each of them. The parallel circuit that is the basis for our move generator is then presented. While this circuit is too large to be built using conventional hardware and has too much communication to allow the use of VLSI, we show how it can be restructured to allow VLSI im-

plementation without sacrificing the performance characteristics of the original circuit. This circuit transformation consists of distributing the state information throughout the circuit instead of sending it from some central location. While the architecture we describe for a move generator solves one special problem, we believe that the same ideas, if not the same architecture, can be applied to other non-numeric computation tasks. Finally, we present the current status of the hardware implementation and discuss performance figures.

2. Chess Move Generation

A move generator is the part of a chess program that is responsible for providing the set of legal moves* available to a side from a given board configuration. The need for a hardware move generator arises only when the search algorithm examines a large number of positions (at least 20,000 nodes per second). Such is the case with the current best chess programs, which all employ the brute-force α - β search algorithm. This algorithm performs a depth-first search of the game tree but is able to avoid searching parts of the tree which it knows cannot affect the outcome. At each position, the algorithm tries a move that it has not already tried for that position. This means that the move generator need only supply *one* move at a time, and that it should remember which moves it has already tried when it returns to the same position. It is also the case that the α - β search can do a better job of pruning the search tree if better moves are tried first. Thus the move generator can support the search algorithm by providing moves in order of *goodness*. While the *a priori* information used to do the ordering cannot predict the true ordering as determined by a search, even a small amount of effort ordering the moves is well spent. If the move generator cannot do move ordering, then either the moves must be tried in an order independent of the current position or all the moves at the position must be extracted from the move generator and ordered externally, a process that would nullify most of the performance of the move generator.

For the move generator to be part of a successful brute-force search, it must be able to perform the following operations efficiently:

- *Update*: Whenever a move is examined the move generator must update its state so that a new chess position can be explored.
- *Backdate*: After all the necessary descendants of a position have been examined the search algorithm will back-up one level in the search tree and continue the

*These are actually pseudo-legal moves since they may include moves which would result in an illegal position, that is, where the move side is in check. We assume that these illegal moves are filtered out by other means and will continue to use the term legal move.

search. The move generator must be able to restore its state appropriately.

- **Move Retrieval:** When called upon, the move generator must provide a move to the controlling program.
- **Move Ordering:** The set of legal moves at a position are ordered using *a priori* information and moves are presented in this order during move retrieval. The amount of information used to do this ordering determines the effectiveness of this ordering.
- **Context Maintenance:** Part of the state of the move generator must be the context of the search for each active position. This context indicates which moves have yet to be retrieved at that position and must be restored during backdate operations.

3. Other Hardware Implementations

The best known hardware move generator is that used in Belle¹, the Bell Labs chess machine that has achieved a master level of play. Belle's move generator supports all the operations described in the previous section. The move ordering is done by virtue of the way in which moves are generated:

- The moves of all the pieces for one side are generated in parallel and the set of destination squares are unioned to yield the set of squares to which at least one piece can move.
- The square containing the most valuable piece of the opposing side is chosen as the destination square of the move. Empty squares are considered to contain pieces less valuable than a pawn.
- The set of pieces that are able to move to this one square is generated.
- The least valuable of these attacking pieces is chosen as the moving piece.

Since each of these operations is done in one cycle, Belle's move generator can very quickly generate what it thinks is the best move. The context of the position is maintained by masking moves as they are generated. A hardware stack maintains the move generator state, including the move masks, during the search.

Although Belle's move generator has excellent performance and provides all the functionality required, there are two drawbacks from our point of view. First, although the ordering of moves is much better than random, the heuristics it uses could be stronger. Moreover, it is not clear whether its scheme is flexible enough to allow moves to be generated in a different order. On the other hand, the simplicity of the ordering greatly facilitates its implementation.

Second, and of greater concern when contemplating using VLSI to implement this type of move generator, is the great amount of communication between the different components. The Belle move generator consists of 64 similar circuits arranged as the squares of a chessboard and there are more than 50 wires between each circuit and its neighbors. In VLSI, the area needed to run this many wires might well be more than the area used by the active circuits. Moreover, since the amount of circuitry required for such a move generator is too much to fit on a single chip the communication required between chips would cause a pinout problem. On the whole, it does not appear that the Belle move generator lends itself easily to VLSI implementation.

Another example of a hardware move generator is one that has actually been designed as a VLSI chip³. This implementation contains the circuitry to calculate the legal moves for a small part

of the board at a time. The board state must be shifted through this functional unit in several major cycles, and several minor cycles are required to extract the legal moves generated during each major cycle. Since there is no support for ordering moves or saving context, all moves must be generated for each position. Thus it takes an average of over 300 machine cycles or 100 microseconds with the stated 3 MHz. clock. Not only is this performance much less than required for a high performance chess machine, the operations that would allow it to be integrated into a chess-playing system are not provided.

4. Design Outline

Our design for a move generator revives an old idea proposed in the early 1960's by Allen Newell. The idea is that there are a fixed number of moves (about 8000) that can ever be possible (we will call this the set of *ever-possible moves*) and the problem is one of determining which moves in this set are actually legal in the current position. A circuit to compute the legal moves in parallel would have as input the current board position and yield as output a vector representing the ever-possible moves, of which only those legal in the current position would be asserted. Each ever-possible move is computed as a simple function of between two and ten inputs of the approximately 250 binary inputs that represent the board position. The result is several tens of thousands of gates connected by about the same number of wires to the board representation inputs. The fanout of the inputs is high (about 100) and uniform, making a partitioning of the circuit very difficult.

This straightforward solution has three drawbacks. First, even by today's standards the amount of circuitry required is large. Second, the communication (wiring) required to connect the inputs to all the function circuits is large. Finally, there is the problem of quickly extracting which outputs are asserted from the 8000 possible moves. The first problem can be solved using VLSI technology where thousands of gates are routinely fabricated on a single chip. But communication is expensive in VLSI where connections must be made in the same plane and using the same material as for the active devices. Long wires can be slow and use chip area that could otherwise be used for devices. Moreover, the packaging technology places severe restrictions on the amount of communication that can take place between different chips.

Our solution to the communication problem is to physically distribute the system state throughout the circuit to the points where it is used instead of trying to communicate the values in parallel from a central location. If a value is used in 100 different places as input to 100 different functions, a copy of the value is kept in each place. This solution relies on being able to quickly change all the copied values as the system changes, which can be accomplished if the change in the system state can be described succinctly (by what we will call *events*) and if only a small number of events take place between the times that the outputs are used. Instead of communicating the system inputs to all the function sub-circuits, only events are communicated.

In chess, the board position changes as moves are made and the output of the move generator is used after each move. All moves (except castling) are accomplished by two events, each represented as only 10 bits. This change in the method of communication changes the large wiring network of the original circuit to a single narrow broadcast bus of only 10 wires. The area used by the extra memory to store the copied values is certainly less than the area required to run the wires in the original circuit, even if it were technically feasible to do so. Figure 1 illustrates how the communication structure is changed by replacing a tangled wiring network by a single event bus. The new circuit is much more regular allowing a straightforward chip layout.

One can also view move generation as a large pattern recognition problem where there are 8000 possible subpatterns to be

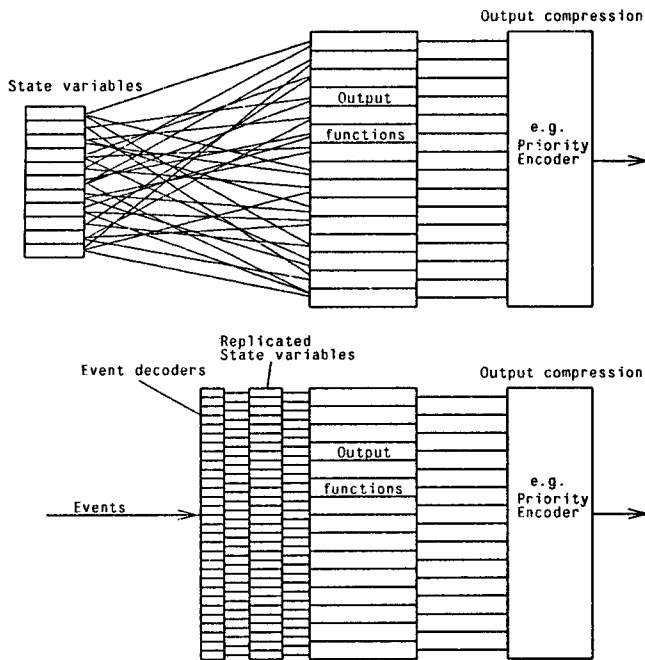


Figure 1: The tangled wiring network that carries the state variables to the function circuits in the first circuit is replaced by a single event bus that communicates the change in the state variables. Copies of the state variables used by each function are kept with the function circuit itself.

found in a board position. Each of the 8000 output functions of the move generator looks for one particular pattern corresponding to one ever-possible move. Each is connected to the event bus and modifies its output based on events that arrive and what it knows about the board state from previous events. The result is the parallel computation of the 8000 move functions with very little communication overhead either in terms of time or chip area.

The final requirement on our parallel circuit is that of extracting the legal moves from the vector of 8000 output signals. There is not much point to computing the set of legal moves in 2 cycles if the representation requires an additional 1000 cycles to reduce it to a more usable form. We use a multi-level priority circuit to quickly determine which move outputs are asserted. This circuit also performs move ordering at the same time using several heuristics. The position context is maintained using on-chip stacks. The result is a move generator that provides all the functionality required by the brute-force search and one that can be implemented using VLSI.

5. Design and Implementation Details

The move generator as described in the overview is much too large to fit on a single chip, especially with the additional circuitry required for the context stack and move ordering. The structure of the problem of move generation was used to partition the circuit into manageable units that could be implemented as chips. Note that the structure of the architecture makes such a partitioning straightforward since there is no communication between functional circuits outside of the move broadcast bus. The set of ever-possible moves is divided according to destination square. Each of the resulting 64 partitions contains about 160 moves and is implemented as a single chip. Since the chips are all computing the same function but for a different square, all 64 chips in a complete move generator are identical and dynamically assigned to one of the 64 squares by setting internal registers.

This partitioning also fits well with the requirements of the move ordering algorithm. Moves are ordered statically within each chip according to the heuristic that says lower valued piece moves should be tried first^{**}. Each chip computes a priority value for the best move (if any) to its assigned square using information about the type of capture and the apparent safety of the destination square. All 64 chips then vote to determine which has the best overall move. Since the safety and capture components do not change the order of moves for a particular destination square but only among destination squares, the overall move ordering is preserved by this partitioning. The structure of the move generator allows this ordering to be done efficiently. While the move ordering done here uses a subset of the information that normal chess programs might use, we chose this since the information is essentially free and more complete information would be difficult to compute.

Each chip computes the legal moves to one destination square. For any given square there exists a maximum of 77 possible moves that can be made to that square ignoring which side is to move. The example shown in Figure 2 enumerates all the possible moves that can be made to the square e4.

8	BQ				RQ			
7		BQ			RQ			BQ
6			BQ	N	RQ	N		BQ
5			N	KP BQ	KP RQ	KP BQ	N	
4	RQ	RQ	RQ	KRQ	*	KRQ	RQ	RQ
3			N	KP RQ	KP RQ	KP BQ	N	
2			BQ	N	P	N	BQ	
1		BQ			RQ			BQ
	a	b	c	d	e	f	g	h

Figure 2: The set of ever-possible moves to a single square

There are 8 king, 27 queen, 14 rook, 13 bishop, 8 knight and 7 pawn moves to the square e4, which makes it one of the few squares to which all the 77 moves are legal. For squares along the edge of the board, the number is much smaller.

For a move to be legal, a small number of conditions must be satisfied:

- The piece making the move must be on the origin square. Thus for the move Q/g6 - e4 to be legal, a queen must be located on the square g6. This is the *origin condition*.
- The destination square must either be empty or contain a piece of the opposite color. This is the *destination condition*.
- For sliding moves (queen, rook, bishop and certain pawn moves^{***}), the squares between the origin and destination must be empty. This is the *sliding condition*.

A block diagram of the move generator chip is given in Figure 3. Each chip contains a set of registers that maintains the information for the origin condition for each move to the destination square represented by the chip. The origin conditions are combined with the sliding conditions to produce the vector of

^{**}This heuristic is similar to one used by Belle.

^{***}The pawn can advance two squares from its initial position.

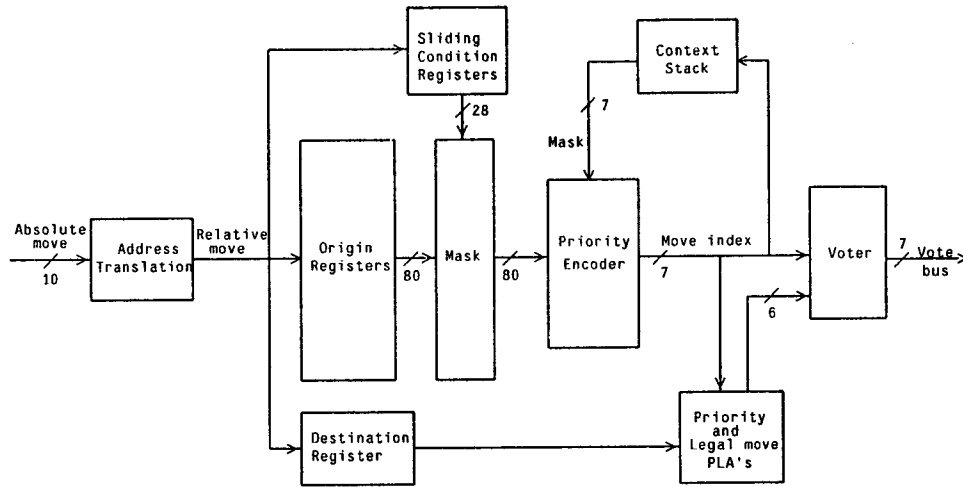


Figure 3: A block diagram of the move generator chip

move signals for each side. These are fed into a priority encoder that yields the value of the highest priority legal move. One PLA calculates whether the move is legal based on the state of the destination register. A second PLA calculates a priority value for the move that is then used to arbitrate between the 64 different chips.

The entire 64 chip configuration is connected as shown in Figure 4.

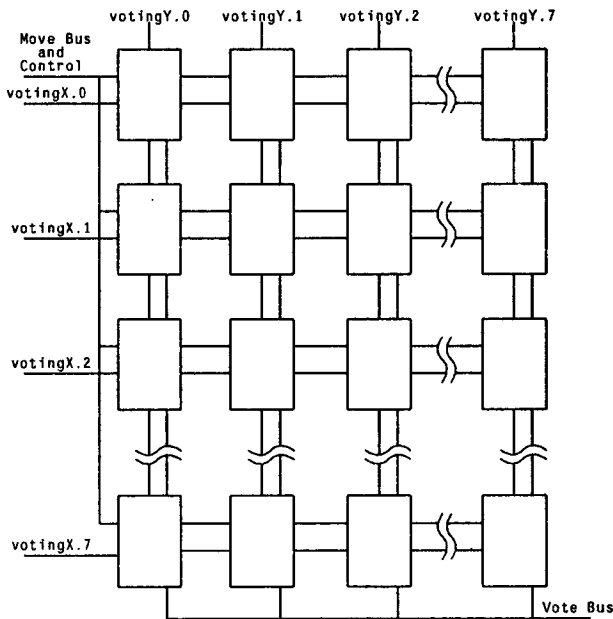


Figure 4: The move generator configured from 64 chips

The move bus communicates moves to the chips as they are made and the vote bus is used for arbitration and to retrieve moves from the chips. The chips are controlled by a command bus which indicates which operation the chips are to perform. These operations include updating, backdating, move retrieval and context handling. Several additional signals are used to select one chip or a set of chips for a particular operation or to modify the operation being performed.

5.1. Move Registers

Each chip contains 80 black/white register pairs that reflect the origin condition for each of the 80 moves**** for which the chip is responsible. An origin register is set if and only if the correct piece is on the correct origin square. Each origin register is addressed in parallel by an internal move bus that broadcasts a half-move which is the move of one piece to one square. A normal chess move can be accomplished by two half-moves. For example, the move *Q/g6 - e4* would be done by the two half-moves {NULL to g6} and {Q to e4}. The second half-move {Q to e4} causes the removal of the piece (if any) that already occupies that square. Note that normal moves can be reversed with two half-moves as well. Figure 5 illustrates how each register pair is addressed. Only the registers that have the same address as on the bus are changed and these are set or cleared depending on whether the piece on the bus matches the register piece address.

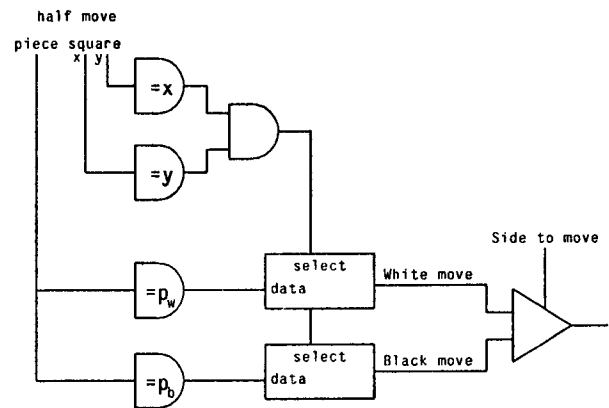


Figure 5: Addressing circuit for the origin registers

**** Earlier we stated that there exists a maximum of 77 possible moves to a given destination square. In calculating that number, we used the fact that there can be only one eight square diagonal and one seven square diagonal through a given square, along with the fact that there is only one possible two square pawn move to a given square. In attempting to design a single chip to handle all squares it was necessary to assume that both diagonals could be of length eight (adding two additional moves, one for the bishop and one for the queen) and that both two square pawn moves could be legal (adding one additional move). Thus, there are 80 move lines of which at most 77 could ever be legal for any given destination square.

The internal move bus is connected to the external move bus through a circuit that converts absolute board addresses to relative addresses. This addressing is discussed in detail in Section 5.3.

Although there are 56 sliding moves, only 28 different sliding conditions need to be calculated since the queen moves share the same origin addresses as the rook and bishop moves. These are used to mask the origin register outputs for sliding moves, enabling only those whose sliding condition are met. Since the calculation of the sliding conditions is complicated by the destination assignment of the chip, its description is deferred to Section 5.3 on addressing.

The destination condition is shared by all 80 moves generated by a chip. A destination register, also addressed by the move bus, keeps track of the state of the destination square. If the destination condition is not met, the chip is prevented from generating any moves by the legal-move PLA.

5.2. Move Ordering and Selection

Each chip independently calculates the set of legal moves to its own destination square as described in the preceding section. Now the best of these moves must be reported to the controlling chess machine. This problem is solved in two stages: first each chip decides which of its moves is best, and then a global voting procedure decides which of the 64 chips has the best move.

The result of the legal move calculation on each chip is a vector of 80 signals. These 80 signals are ordered according to the heuristic that moving lower valued pieces to a given square should be tried before moving higher valued pieces. These are fed into a 80 to 7 priority encoder that outputs the index^{*****} of the highest priority legal move. Each chip then calculates a 6 bit priority value based on the type of the moving piece, the type of the captured piece, if any, and an estimate of the safety of the destination square. The safety is determined first by which side has pawns guarding the square and second by the next lowest valued piece guarding the square for each side. For example, the priority value of a favorable capture will be lessened if the square is guarded by the opponent's pawn. Unfortunately, the corresponding safety information about the origin square can not be made available to the destination square in an efficient manner, but this information is generally less important.

To determine which chip has the highest priority value, inter-chip voting is performed using the priority value each chip has calculated; chips that have no legal moves do not participate. To eliminate ties, each chip concatenates the 6 bit priority value it has calculated with a unique 6 bit id assigned to it at initialization. The chips first vote on the high order 3 bits of the priority using a global 7 bit wired-OR voting bus. If there is a tie, then the chips still tied vote on the next 3 bits of the priority. This process continues until only one chip is voting. The controller senses when only one chip is still voting using two sets of wired-OR signals, one connecting the chips vertically and the other horizontally as shown in Figure 4. Chips still participating in the vote assert both wires and if only one wire in each set is asserted, then only one chip is voting and no more voting cycles are necessary. This chip is then instructed to place the index of its move on the voting bus.

This selection mechanism is able to produce the first move in the list of legal moves to each destination square, but there must be some way to produce the rest of the moves to the square. That is, the context of the position must be saved and restored and the selection process must be able to use this context to give the remaining moves.

^{*****} Each move to a destination square is assigned an index from 1 to 80 in order from high to low priority. Given the index and the destination square number it is possible to uniquely determine the actual move retrieved.

The index of the previous move generated is sufficient context for each destination square. Given this index, the priority encoder produces the next highest legal move index. This is done by augmenting the priority encoder with the ability to mask all moves less than a given index. A mask value of N disables the first N inputs to the encoder. Given that move indices range from one to 80, the index of the last move retrieved from a chip can be used as the mask to the priority encoder to obtain the next move. If no moves have yet been generated, the mask is set to zero.

The state that must be saved until the next move is requested consists of the board state contained in the origin, sliding, and destination registers plus the index of the previous move and the safety information. The board state can easily be restored by reversing moves made when going forward in the search tree. A 40 element stack is provided for the move index and safety information. All the chips push and pop this stack simultaneously as the search is performed, but only the chip whose move is actually used updates the index to the new move while the others push the previous index.

5.3. Chip Addressing and Square Assignment

Although each of the 64 chips in the move generator calculate the moves to a different square, they are identical in design and individually assigned to different destination squares. This is accomplished by loading special registers on each chip with the description of its intended location. The first of these registers is the Destination Address Register which is used to convert all square addresses from absolute addresses to addresses relative to the destination square, whose relative address is (0,0).

Figure 6 gives a picture of the chess board as seen by each chip. The numbers indicate the addresses of each square relative to the destination square in the center, and the letters represent the possible moves from the squares. Figure 7 shows two different chips assigned to different locations on the board. For any destination square assignment, many of the origin registers represent origin squares that lie off the board. Since the addresses for these squares are never generated, these register are idle. Although this waste of registers cannot be avoided for the pawn, knight and king registers, the number of registers for the sliding pieces is reduced by assigning each register to two squares, only one of which lies on the board. This is done by modifying the addresses of the ray registers as shown for the the horizontal ray in Figure 8. Each register responds to moves to either of its assigned squares but since only one square is actually on the board, no ambiguity arises.

The computation of the legality of a sliding move must calculate whether there is an open path from the source to the destination square. This calculation determines the way the piece must move to reach the destination square depending on the destination assignment of the chip and checks whether the intervening squares are empty. The direction information is derived from the destination square assignment, and the path information is contained in set of ray registers that indicate whether the squares on the rays are occupied. These registers are the same as the origin square except registers except that they are set if the square is occupied and cleared otherwise.

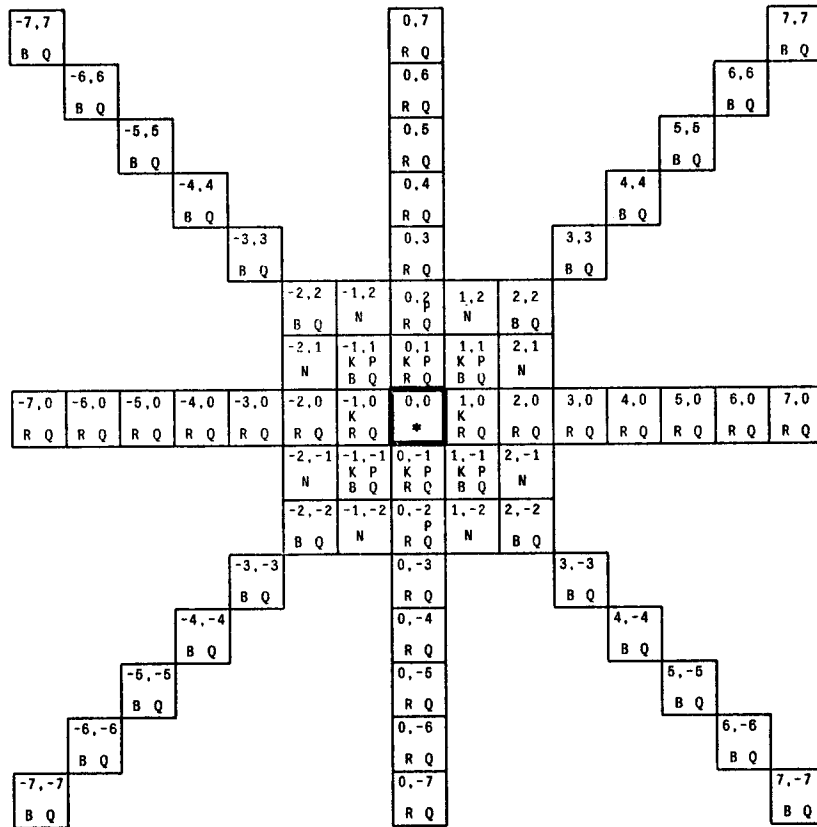


Figure 6: View of chess board from one chip

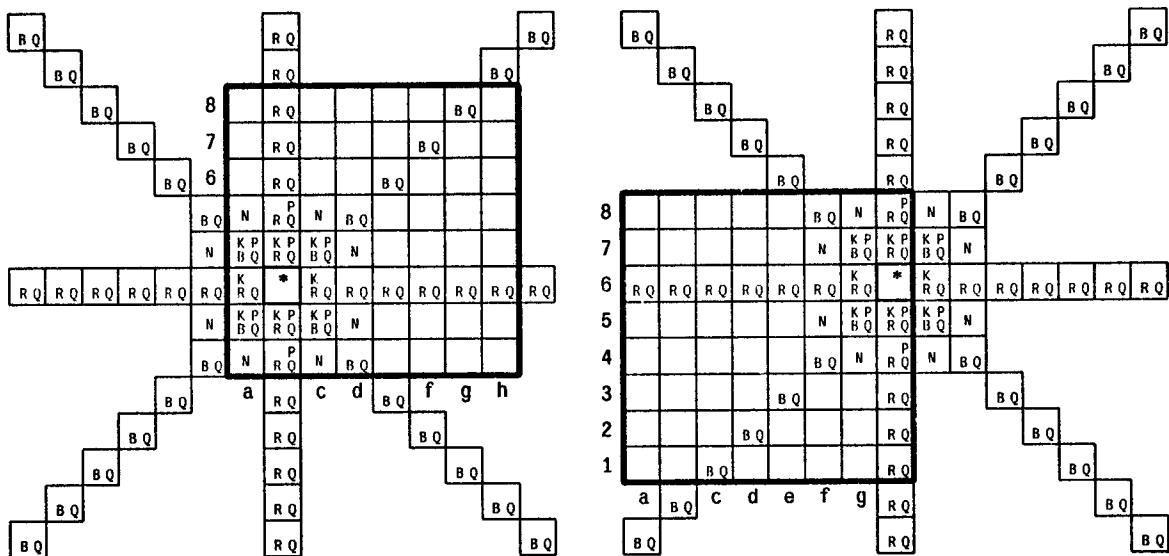


Figure 7: The view of two chips assigned to different destination squares

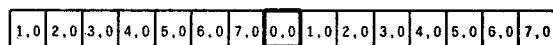


Figure 8: The relative addresses of ray squares is modified so that each register is assigned two address. Since only one of the two addresses can be generated, it is clear which move the origin register represents.

6. The Operation of the Move Generator

Figure 9 summarizes the operation of the move generator both when extending the search tree by making a move to a new position and when returning to a previous position. Each operation is executed during a single clock cycle with a five bit command word specifying the actual operation to be performed. The set of chips executing the operation is selected using four different chip select signals. These are normally wired so that a single chip, one ray of chips, or all the chips can be selected. The chip select signals are also used to modify the push stack operation so that only one chip pushes a new move index while the other chips push the previous one. An additional control signal is used to specify that only captures are to be generated which is used during the quiescence part of the search.

All chip registers are linked together as part of a shift register that is also controlled by the command word. This shift register is used to initialize the move generator to the initial configuration and to access the internal state of the chips during testing and debugging. All registers are static during one phase of the clock so that the move generator can be halted between any two operations.

Update to next position

Operation	Output signals
Push context stack	
Execute white half-move	
Execute white half-move	Black in Check?
Latch Black's best move	White in Check?
Compute priority value	
Vote cycle 0	Voting X/Y
Vote cycle 1	Voting X/Y
Vote cycle 2	Voting X/Y
Vote cycle 3	Voting X/Y
Output Move	Move index

Backdate to previous position

Operation	Output signals
Pop stack	
Execute reverse white half-move	
Execute reverse white half-move	Black in Check?
Latch Black's next move	White in Check?
Compute priority value	
Vote cycle 0	Voting X/Y
Vote cycle 1	Voting X/Y
Vote cycle 2	Voting X/Y
Vote cycle 3	Voting X/Y
Output Move	Move index

Figure 9: The execution sequence of the move generator

7. Project Status

The move generator chip was designed, fabricated, and tested over the past two years. The final version of the chip is now being fabricated and we expect to begin construction of the entire 64 chip move generator during Spring, 1984. The chip was designed using Mead-Conway design rules and has been fabricated through the DARPA MOSIS fabrication service using a standard NMOS process with lambda of 2 microns. The chip measures 6800 x 6300 microns and is packaged in a standard 40 pin DIP.

Measurements of first silicon indicate that the move generator will operate well within a 250ns cycle time, resulting in 2 microseconds per move if an average of 8 operations are required for each position. This is comparable to the move generator used by Belle and given the improved move ordering mechanism used by our design even greater speedup in overall performance is expected. It should be noted that the performance of this move generator derives from the parallel architecture and not from the VLSI implementation since we are using rather unambitious circuit design and technology. The real role of VLSI here was in allowing the architecture to be implemented at all.

This move generator is the first component of a planned chess machine. Design of this chess machine is currently underway and a minimal chess machine consisting of a α - β controller, the move generator, a simple evaluation function, and a transposition hash table should be completed during Summer, 1984. At that time we intend to investigate how well the ideas we have used in designing a parallel move generator apply to a chess position evaluator. We have already determined that the basic design used by the move generator can be used for detecting a number of features, including control of squares and pinned pieces.

8. A Final Comment

We have described a design that represents a departure from conventional designs which do communication in a more obvious way. In this design, communication was greatly reduced by replicating information throughout the circuit allowing a straightforward VLSI implementation. This idea may be applicable when using VLSI for other large parallel tasks, especially within the domain of Artificial Intelligence. One of the major problems facing large Artificial Intelligence systems is the ability to integrate large amounts of information. Our research demonstrates one area where by replicating information, large amounts of processing can be done in parallel with minimal communication costs.

This research was supported in part by the Defense Advanced Research Projects Agency (DoD), ARPA Order No. 3597, monitored by the Air Force Avionics Laboratory under Contract F33615-81-K-1539.

References

1. Condon, Joseph, and Thompson, Ken, "Belle chess hardware," in *Advances in Computer Chess III*, University of Edinburgh Press, 1981.
2. Knuth, D.E., and Moore, R.W., "An Analysis of Alpha-Beta Pruning," *Artificial Intelligence*, Vol. 6, No. 4, 1975, pp. 293-326.
3. Schaeffer, Jonathan, Powell, Patrick A.D., and Jonkman, Jim, "A VLSI Chess Legal Move Generator," *Proceedings of the Third Caltech Conference on Very Large Scale Integration*, Bryant, R., ed., Computer Science Press, Inc., California Institute of Technology, March 1983, pp. 207-223.